Transportation Research Part C 191 (2026) 105824



Contents lists available at ScienceDirect

Transportation Research Part C

journal homepage: www.elsevier.com/locate/trc





# A scalable reinforcement learning-based approach to dynamic airspace sectorization

Wenxuan Wang *, Arnab Majumdar, Washington Ochieng, Jose Escribano

*Department of Civil and Environmental Engineering, Imperial College London, London, SW7 2AZ, London, United Kingdom*

ARTICLE INFO

*Keywords:*
Dynamic airspace sectorization
Reinforcement learning
Air traffic controllers' workload
Tiled Voronoi partitioning

ABSTRACT

Dynamic airspace sectorization (DAS) is a key enabler of demand and capacity balancing (DCB) in air traffic management. However, existing DAS methods face significant limitations in computational efficiency and scalability when applied to real-world operations with complex traffic patterns and varying sector boundaries. This study addresses these limitations by formulating DAS as a Markov decision process and developing a novel reinforcement learning (RL) framework that generates optimized sectorization sequences through policy-driven sequential decision-making. Unlike previous DCB studies that focus primarily on traffic flow management, this work represents the first RL framework developed for the DAS problem. The proposed framework integrates three-dimensional tiled Voronoi partitioning with actor-critic RL to dynamically adjust sector boundaries in response to predicted traffic demand. A comprehensive workload model is developed that considers aircraft proximity at sector boundaries, sector crossing frequency, sectorization similarity, and workload imbalance across sectors. The framework is trained and validated using three weeks of historical ADS-B trajectory data from UK airspace, encompassing over 200,000 flights across different seasonal periods. Experiments demonstrate significant improvements over a state-of-the-art genetic algorithm (GA) benchmark: an average workload reduction of 50.59% and a decision-making time reduction of nearly 99.98% (from 82.22 minutes to 1.31 seconds per interval). These results highlight the potential of the proposed framework for real-time DCB and its broader applicability to intelligent air traffic management systems. The complete codebase is publicly available at: https://github.com/Haileyww/RL_DAS.

## 1. Introduction

Air Traffic Management (ATM) in Europe faces severe congestion during certain peak time intervals (Nosedal et al., 2015; Mannino et al., 2021). The Single European Sky ATM Research (SESAR) has also highlighted the mismatch between traffic demand and available ATM capacity as the primary challenge and has launched a project to overcome the limitations on airspace capacity by modernizing and restructuring the European ATM system (Bolić and Ravenhill, 2021; Delgado et al., 2021).

Air Traffic Controllers' (ATCo) workload, which generally refers to the cognitive demands placed on controllers to meet the requirements of their tasks, is widely recognized as a key factor limiting ATM capacity (Chaloulos et al., 2010). This mental demand encompasses human factors such as fatigue and stress, rather than just physical effort or the time required to complete a task. Measurement approaches for ATCo workload can generally be classified into three categories: performance-based, subjective, and

* Corresponding author.
*E-mail addresses:* wenxuan.wang22@imperial.ac.uk (W. Wang), a.majumdar@imperial.ac.uk (A. Majumdar), w.ochieng@imperial.ac.uk (W. Ochieng), j.escribano@imperial.ac.uk (J. Escribano).

https://doi.org/10.1016/j.trc.2026.105824
Received 11 December 2025; Received in revised form 11 June 2026; Accepted 19 June 2026
Available online 29 June 2026
0968-090X/© 2026 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

physiological measurements (Yazgan et al., 2021). Performance-based approaches quantify ATCo workload solely through flight data, such as the number of clearances issued (Tobaruela et al., 2014) and trajectory characteristics, without requiring physiological tracking or survey questionnaires. This makes performance-based methods particularly feasible and convenient for airspace sectorization optimization models. Therefore, most research on airspace sectorization optimization—primarily employing evolutionary algorithms—adopts performance-based workload metrics to minimize ATCo workload.

While flight trajectory optimization could theoretically reduce airspace congestion and ATCo workload (Djokic et al., 2010), trajectories are planned by airspace users (e.g., airlines) to minimize their own operational costs, and airlines have no direct incentive or visibility to optimize for ATCo workload. This makes trajectory-based workload reduction infeasible as a practical capacity improvement strategy from the perspective of an Air Navigation Service Provider (ANSP) (Xu et al., 2020a). Instead, the main mechanisms used by ANSPs when demand exceeds capacity involve either ground delays or route restrictions. However, each of these strategies has its own drawbacks. Ground delays are economically costly (Bolić et al., 2017), while route restrictions may impose additional fuel penalties on airlines; both manage demand but do not increase capacity. In contrast, airspace sectorization restructures the current airspace to better match capacity with air traffic patterns. Consequently, recent research has emphasized methods to optimize airspace sectorization in order to increase airspace capacity.

Growing air traffic demand over recent decades has motivated the development of dynamic airspace management approaches: Dynamic Airspace Configuration (DAC) (Kopardekar et al., 2007) and Dynamic Airspace Sectorization (DAS) (Kopardekar et al., 2008). DAS/DAC methods are of strong practical interest because they adapt sector configurations to match temporal traffic variations, thereby improving airspace capacity utilization. Static sectorizations have remained relatively constant, while route structures and demand have changed dramatically over the years. Thus, SESAR formally recommends DAC as the primary means to resolve demand-capacity imbalances (Lui et al., 2024). Some ANSPs already practice limited dynamic reconfiguration—opening and closing sectors during peak hours to match traffic demand, even if the degree of flexibility varies across ANSPs. DAS/DAC also sits within the UK airspace change masterplan, the objective of which is airspace modernization with reduced constraints on sectors' ability to manage traffic flows safely (Zhao et al., 2026).

The two approaches differ in their partitioning methodology: DAC reconfigures airspace by grouping predefined two- or three-dimensional cells into sectors, whereas DAS directly adjusts sector boundaries without cell-based constraints (Schultz et al., 2019). Our proposed framework follows the DAS paradigm, generating new sectorizations by directly optimizing boundary placement at each time period, without relying on predefined cells or incremental split/merge operations. DAS has an advantage over DAC in guaranteeing convex sector shapes. DAC operates on pre-designed quadrilateral cells, which can result in fragmented sectors unless additional convexity constraints are imposed, thereby limiting design flexibility. However, DAC offers lower operational complexity as it operates on fixed cells, whereas DAS introduces more boundary changes that may increase ATCo adaptation workload, as controllers must re-familiarize themselves with modified sector geometries. Additionally, as noted in Hind et al. (2018), the benefits of DAC often require post-processing steps to be fully realized.

Despite its potential, wider adoption of DAS/DAC is currently limited by several obstacles. On the technical side, existing DAS/DAC solvers lack efficient mechanisms for predicting workload and optimizing re-sectorization in real time. While they successfully reduce ATCo workload, they require long computation times, limiting their real-time applicability (Sergeeva et al., 2017; Wong et al., 2020). Furthermore, these approaches optimize each time period independently, preventing them from balancing immediate workload reduction against long-term operational continuity. On the human-factors side, frequent reconfiguration imposes cognitive challenges on ATCos (Tang et al., 2012). When addressing this additional source of workload, existing methods mostly consider it separately during post-processing, leaving the overall total ATCo workload suboptimal. Addressing these two main obstacles—particularly the computational challenge—is therefore a prerequisite for practical implementation.

Therefore, this paper explores and evaluates the development of a reinforcement learning (RL)-based framework for DAS problems: it simulates a mathematical environment that replicates the structure of UK airspace and incorporates comprehensive historical flight trajectory data. We adopt UK airspace as the case study because it is among the busiest and most complex in the world, making it a demanding and realistic testbed. Within this environment, we formulate the re-sectorization process as a Markov decision process (MDP) and train the DAS solver using RL algorithms tailored to various flight patterns. By modeling re-sectorization as a continuous process, our approach can effectively balance the similarity between consecutive sectorizations and the required adaptations for varying traffic in subsequent time intervals. Additionally, the trained solver can generate new sectorizations immediately in response to real-time changes. This paper also outlines the limitations of this approach—such as large data requirements, extended training time, and potential instability. To summarize, the main contributions of this study are as follows:

1. This study develops a novel RL-based framework that generates re-sectorization decisions in near real-time. The framework employs Voronoi-based sector generation to guarantee convex sector geometries, supporting practical operational feasibility.

2. The framework adaptively adjusts current sectorization with a built-in sector similarity metric, eliminating the need for post-processing while maintaining operational continuity.

3. The framework employs a historically-grounded training paradigm, learning from historical real-world UK airspace trajectories and validating on future flight plans, demonstrating its applicability to large-scale, high-density air traffic environments.

4. Experimental results demonstrate that the proposed method outperforms state-of-the-art benchmarks in solution quality, computational efficiency, and scalability for complex traffic patterns.

The remainder of this paper is organized into six sections. Section 2 examines related research on the state-of-the-art DAS problem. Section 3 explains the MDP formulation of the DAS problem and the modeling of objectives. Section 4 explains the proposed RL framework and the use of a tiled Voronoi diagram. Section 5 presents the experimental setup, including data preparation and the

fine-tuning process. Section 6 presents the numerical results of the experiments and the comparison between different RL techniques and existing methods. Finally, Section 7 closes this paper with the contributions of this work and future directions for research.

## 2. Literature review

This section reviews related work on airspace sectorization optimization and RL techniques that inform the methodology developed in this study.

### 2.1. Studies on airspace sectorization

Research on airspace sectorization/configuration optimization has evolved over recent decades to address the growing demand for air traffic. Early studies focused primarily on static sectorization/configuration using evolutionary approaches, such as Delahaye et al. (1998). Most airspace sectorization approaches incorporate Voronoi diagrams to ensure sector convexity, whereas airspace configuration methods typically employ clustering algorithms to group predefined cells.

Studies in the 2000s began exploring DAS/DAC concepts but were limited by simplified workload models and constrained flexibility. For example, Martinez et al. (2007) applied a spectral bisection method to partition weighted graphs into sector subgraphs but relied solely on traffic counts as a performance metric without a comprehensive ATCo workload model. Addressing limitations like this, Tang et al. (2012) proposed a multi-objective framework combined with agent-based modeling to better capture ATCo workload.

With advances in metaheuristic algorithms, Genetic Algorithm (GA)-based approaches to DAS/DAC have become increasingly prevalent in recent years, accompanied by more sophisticated ATCo workload models. Chen et al. (2013) used an improved GA for dynamic sectorization. Similarly, Sergeeva et al. (2017) refined the weighted graph model of Martinez et al. (2007) and integrated it with GA to generate dynamic solutions. Gerdes et al. (2016) designed a three-step flight-centric DAS solver—later refined in Gerdes et al. (2018)—that combines fuzzy clustering for flight grouping, Voronoi diagrams for sector generation, and evolutionary optimization. This work also introduced a doubly connected edge list developed in De Berg et al. (2008) to store the merged structure of the Voronoi diagram and airspace borders. Although this study analyzed area changes between sequential sectors, these variations were not incorporated into the optimization objective. Subsequently, Schultz et al. (2019) developed an enhanced DAS framework with a similar architecture but distinguished by explicit ATCo task load modeling and operational validation.

Wong et al. (2018) aimed to minimize ATCo workload imbalances, coordination workload, and sector dissimilarity. Unlike previous studies, it explicitly addressed the conflict between monitoring workload and coordination workload. This study applied a non-dominated sorting GA combined with a Voronoi diagram, followed by a meta-cognitive decision maker that selected solutions satisfying constraints and predicted traffic. It quantified sector similarity using the ratio of the maximum overlap area to the previous sector area. However, this method only accounts for the largest sector size change, overlooking smaller changes in other sectors and the impact of shape variation. The study evaluated its approach by using Singapore airspace and historical flight data.

Wong et al. (2020) proposed a DAS approach using a rolling horizon GA and a Voronoi diagram to generate sectorization for three consecutive time periods simultaneously. Its objectives include aircraft density (representing monitoring workload), the number of flights crossing sector boundaries (representing coordination workload), and sector overlap (representing sector similarity). This study adopted the same sector similarity metric as Wong et al. (2018) and consequently shares its limitations.

Xu et al. (2024) presented a DAS model incorporating RL-based workload prediction techniques. It generated sector boundaries using constrained k-means clustering and a support vector machine algorithm. This paper focused on ATCo workload prediction and adopted a workload measurement based on flight tasks, as developed in Tobaruela et al. (2014). Meanwhile, it does not consider sector similarity in the optimization process, nor does it include a subsequent re-sectorization step.

With the proposal of the demand and capacity balancing (DCB) concept, DAS has become an essential component of the process, which adjusts both capacity (typically through dynamic sectorization) and demand (typically through trajectory modification). However, capacity adjustment has been studied and optimized less frequently (Chen et al., 2025). Melgosa et al. (2025) improved complexity metrics and developed a network manager responsible for determining airspace sectorization. Liu et al. (2025) reached a similar conclusion, proposing a multi-objective complexity model for strategic deconfliction. Xu et al. (2020b) introduced an integrated model that employed a simplified DAS approach based on the merging and collapsing of elementary sectors. Their study also highlighted that re-designing elementary sectors can largely increase the computational burden within the DCB process and thus is rarely adopted.

In summary, existing DAS approaches face three key limitations: sector dissimilarity, adaptation continuity, and response efficiency remain insufficiently addressed. These limitations also hinder the development of DCB systems capable of adapting to dynamic traffic conditions. To address these gaps, this study proposes an RL framework that fundamentally differs from existing approaches by formulating DAS as a sequential decision-making problem.

### 2.2. Studies on reinforcement learning

RL problems are typically modeled as MDPs, formally defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the set of possible states, $\mathcal{A}$ is the set of actions, $\mathcal{P}$ represents the transition probability between states, $\mathcal{R}$ is the reward function, and $\gamma$ is the discount factor. This process allows the agent to learn a policy that maps each state to an optimal action by interacting with the environment and maximizing expected cumulative rewards over time.

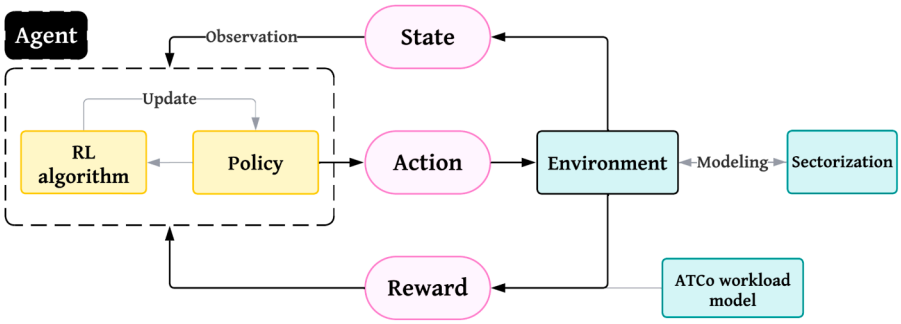



**Fig. 1.** Markov decision process formulation for dynamic airspace sectorization.

RL algorithms are broadly categorized into value-based, purely policy-based, and actor-critic approaches. Value-based methods, such as Q-Learning and Deep Q-Networks, learn to estimate the expected return of state-action pairs and select actions accordingly, while purely policy-based algorithms directly optimize the policy without a value function. Actor-critic RL algorithms, such as Proximal Policy Optimization (PPO) (Schulman et al., 2017) and Soft Actor-Critic (SAC) (Haarnoja et al., 2018), combine a policy network (actor) that outputs actions with a value network (critic) that evaluates them, offering better stability and performance in continuous or high-dimensional action spaces. Notably, actor-critic algorithms are able to handle continuous sector centroid adjustments and are compatible with parallelized environments.

RL involves an agent interacting with an environment by performing actions and receiving feedback (reward/punishment) based on the outcomes of those actions. The agent continuously refines its policy through learning, aiming to maximize the computed reward (Sutton et al., 1998). Although standard RL frameworks assume stationary transition dynamics, recent studies have shown that RL algorithms remain effective in non-stationary environments by adapting policies through continued learning (Padakandla, 2021). Metelli (2022) provided a comprehensive review of such configurable RL frameworks, which are particularly relevant to DAS, where flight patterns—part of the environment—are actively adjusted. The learning mechanism makes RL algorithms particularly well-suited for complex sequential decision-making problems, such as the DAS problem. Unlike traditional methods that solve each re-sectorization separately and rely on post-processing to enforce temporal continuity constraints, an RL agent learns policies that directly optimize cumulative rewards across the entire time horizon. This enables the agent to naturally balance immediate workload reduction with long-term operational stability, incorporating sectorization similarity as a core component of the learned policy rather than a post-hoc constraint.

RL has demonstrated effectiveness across various spatial decision-making problems, such as vehicle routing (Zhao et al., 2020) and logistics optimization (Hu et al., 2025). Recent developments in the ATM domain have also demonstrated the potential of machine learning in addressing complex tasks (Dhief et al., 2022), including flow management (Wu et al., 2021), ATCo workload prediction (Xu et al., 2024), capacity forecasting (Zhou et al., 2023), and DCB (Chen et al., 2023). However, RL has not been applied to DAS despite its suitability for sequential spatial optimization. This study presents the first RL-based DAS framework.

## 3. Problem formulation

This study models the entire sequence of re-sectorizations as an MDP, as shown in Fig. 1. To capture temporal variations in air traffic, an operating day is divided into consecutive time periods of fixed duration, at the beginning of which sector boundaries are evaluated and potentially adapted. In this MDP formulation, a reward function defines the optimization objectives, and an RL agent is trained to maximize the expected cumulative reward through iterative interactions. This section introduces the MDP formulation and reward function components.

### 3.1. Markov decision process

The DAS problem is formulated as an MDP where each re-sectorization at time $t$ depends on the previous sector structure at time $t-1$. Let $\mathcal{O}$ denote the observation space and $\mathcal{A}$ denote the action space. The agent maintains a stochastic policy $\pi : \mathcal{O} \to \mathcal{P}(\mathcal{A})$, which maps each partial observation $o_t \in \mathcal{O}$ at time step $t$ to a probability distribution $\mathcal{P}(\mathcal{A})$ over the action space $\mathcal{A}$, from which the action $a_t \in \mathcal{A}$ is sampled at each time step $t$.

The agent's task is to identify the optimum policy $\pi^*$ that produces the set of actions (re-sectorization) to yield the maximum cumulative reward. The reward $\mathcal{R}(s_t, a_t, s_{t+1})$ for each action $a_t$ is computed based on the resulting state $s_{t+1}$. The cumulative reward $\sum_{t=0}^{\infty} \mathcal{R}(s_t, a_t, s_{t+1})$ is the sum of per-step rewards $\mathcal{R}(s_t, a_t, s_{t+1})$ over the episode. The policy $\pi$ is iteratively updated according to the cumulative reward. The policy optimization process, defined from the current timestep $t$, is given in Eq. (1), where $\gamma \in [0, 1)$ is the discount factor that ensures the convergence of the infinite sum, and $\mathbb{E}$ denotes the expected value of the cumulative discounted

reward from time $t$ onward.

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{s_t, a_{t'} \sim \pi(\cdot|o_{t'})} \left[ \sum_{t'=t}^{\infty} \gamma^{t'-t} \mathcal{R}(s_{t'}, a_{t'}, s_{t'+1}) \right] \tag{1}$$

### 3.2. Reward function

The reward function encodes the DAS optimization objectives. In this study, the reward function is designed to minimize ATCo coordination workload and sectorization dissimilarity. Accordingly, the reward evaluates each action $a_t$ based on four penalty components: proximity events between aircraft near sector boundaries, which constitute the primary source of coordination workload; the number of sector boundary crossings by aircraft; the dissimilarity between consecutive sectorizations, which affects ATCo adaptation complexity; and workload imbalance across sectors. Since each workload component directly quantifies a countable operational event, a reduction in these absolute counts directly corresponds to a reduction in the coordination tasks that ATCos are required to perform. The $c$-th component $\mathbf{W}_c$ is normalized, weighted by its coefficient $w_c$, and summed to compute the total reward $\mathcal{R}$ (with penalties represented as negative values) resulting from transitioning into state $s_{t+1}$, as formulated in Eq. (2).

$$\mathcal{R}(s_t, a_t, s_{t+1}) = \sum_{c=1}^{4} w_c \cdot \mathbf{W}_c(s_{t+1}) \tag{2}$$

ATCo workload is typically categorized into conflict workload, monitoring workload, and coordination workload. In our reward function, monitoring workload is excluded because the total monitoring demand across the entire airspace is determined solely by traffic volume and remains invariant to sectorization design. In contrast, the coordination workload is significantly influenced by sectorization design. As noted in Wong et al. (2018), it is impractical to minimize both simultaneously.

#### 3.2.1. Aircraft proximity at sector boundaries

Avoiding aircraft conflicts is a critical responsibility for ATCos, and managing the proximity between aircraft significantly increases their cognitive load, potentially impairing performance (Suarez et al., 2014). When such proximity events occur near sector boundaries, controllers from both adjacent sectors must monitor the aircraft and coordinate any necessary detours. Therefore, the reward metric $\mathbf{W}_1$ computes the number of proximity events between aircraft pairs occurring near sector boundaries. As shown in Eq. (3), where $n_t$ is the total number of flights during the $t$-th interval, $f_p$ denotes the $p$-th flight, and $f_q$ denotes the $q$-th flight, a proximity event is counted when the horizontal distance $d_{f_p,f_q}$ between flight $f_p$ and $f_q$ is within $\delta_1$ nautical miles (nm), flight $f_p$ is within $\delta_2$ nm of a boundary $b$ in the set of all boundaries $\mathcal{B}$, and both $f_p$ and $f_q$ are at the same flight level (FL): $L_{f_p} = L_{f_q}$. $\mathbf{1}(\cdot)$ is the indicator function that equals 1 if the condition is true and 0 otherwise. The flight level condition ensures that only aircraft with no vertical separation are considered, as vertically separated aircraft do not impose coordination workload on ATCos, regardless of their horizontal proximity.

$$\mathbf{W}_1 = \sum_{p=1}^{n_t} \sum_{q=1}^{n_t} \sum_{b \in \mathcal{B}} \mathbf{1}\Big( \Big( d_{f_p,f_q} \le \delta_1 \Big) \wedge \Big( d_{f_p,b} \le \delta_2 \Big) \wedge \Big( L_{f_p} = L_{f_q} \Big) \Big), \quad p \ne q \tag{3}$$

#### 3.2.2. Sector entries and exits

The second weighted indicator of ATCo coordination workload is the total number of aircraft entries and exits across all sectors. Both an entry and an exit represent transitions between two sectors, requiring a handover between the ATCos of the corresponding sectors and thus contributing to the coordination workload. As computed in Eq. (4), our approach detects sector crossings by checking whether consecutive $e$-th and $e+1$-th trajectory vertices $v_e^{f_p}$ and $v_{e+1}^{f_p}$ of flight $f_p$ belong to different sectors $S(v_e^{f_p})$ and $S(v_{e+1}^{f_p})$. $V_{f_p}$ denotes the total number of vertices in flight $f_p$'s trajectory.

$$\mathbf{W}_2 = \sum_{p=1}^{n_t} \sum_{e=1}^{V_{f_p}-1} \mathbf{1}\Big( S\Big(v_e^{f_p}\Big) \ne S\Big(v_{e+1}^{f_p}\Big) \Big) \tag{4}$$

#### 3.2.3. Sectorization similarity

A challenge introduced by DAS is the increased complexity of sectorization changes. Therefore, nearly all the DAS research involves controlling the extent of the changes during adaptation. As mentioned earlier, current sector similarity measurements are mainly limited to changes in sector area. However, two sectors with the same area but different shapes or flight paths can result in significantly different workload complexity for ATCos. To address this limitation, we develop a sector similarity metric that evaluates the complexity of sectorization adaptation from the perspective of ATCo workload. Rather than solely relying on sector geometric differences, we shift our focus to the changes in the flights that an ATCo is responsible for managing.

As formulated in Eq. (5), where $S_t(f_p)$ denotes the sector managing flight $f_p$ at time $t$, and $S_{t+1}(f_p)$ denotes the sector managing flight $f_p$ after re-sectorization at time $t+1$, we calculate the number of flights that will be newly assigned to another sector or removed from it due to re-sectorization. A flight $f_p$ contributes to $\mathbf{W}_3$ if it is assigned to a different sector due to the re-sectorization at time $t+1$.

$$\mathbf{W}_3 = \sum_{p=1}^{n_t} \mathbf{1}\big( S_t(f_p) \ne S_{t+1}(f_p) \big) \tag{5}$$

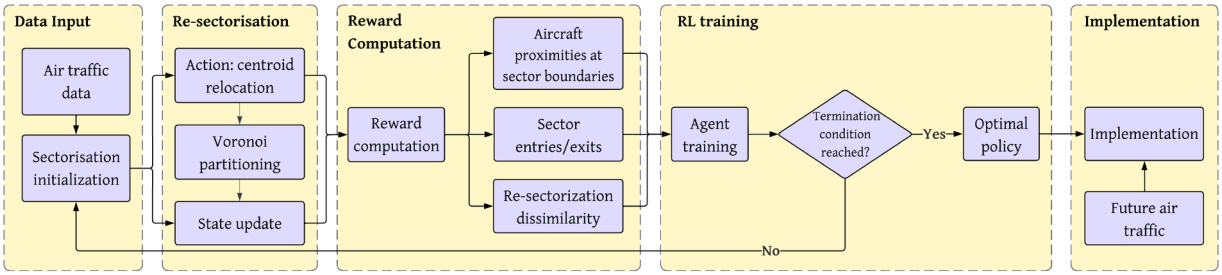



**Fig. 2.** Reinforcement learning framework for dynamic airspace sectorization.

It is worth noting that $\mathbf{W}_3$ evaluates how many flights are reassigned to a different sector purely as a result of the boundary change. Since re-sectorization is completed within seconds, flight transitions during reconfiguration are ignored, providing a direct measure of the cognitive adaptation workload imposed on ATCos by the re-sectorization itself.

#### 3.2.4. Workload imbalance

After establishing an ATCo workload model, the minimization of workload imbalance across sectors is incorporated into the reward function. Our approach utilizes the workload model defined in the reward function to calculate workload imbalance. After a re-sectorization, our approach calculates the total workload for each sector by aggregating the sector-specific contributions from aircraft proximity near boundaries, sector crossings, and sector dissimilarity. The mean sector workload $\bar{w}$ is defined in Eq. (6), where $N$ is the number of sectors and $\mathbf{W}_c^i$ denotes the $c$-th workload component for sector $i$.

$$\bar{w} = \frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{3}\left(\mathbf{W}_c^i\right) \tag{6}$$

Based on this, Eq. (7) computes the standard deviation of ATCo workload $\mathbf{W}_4$ across all $N$ sectors. $\mathbf{W}^i = \sum_{c=1}^{3}\mathbf{W}_c^i$ is the total workload of sector $i$, which is then incorporated into the objective function.

$$\mathbf{W}_4 = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{W}^i - \bar{w}\right)^2} \tag{7}$$

### 3.3. Constraints

In addition to basic operational constraints, two other constraints are implemented in our DAS solver: sector convexity and maximum sector workload.

Sector convexity is guaranteed by construction, as sector boundaries are generated using Voronoi diagrams, which produce convex cells by their geometric definition.

Maximum sector workload is enforced as a soft constraint through the workload imbalance component of the reward function (Section 3.2.4), which assigns higher penalties to sectors with outlying workload values. This formulation allows the solver to balance workload distribution while maintaining flexibility when strict workload limits cannot be satisfied. Unlike existing DAS optimization approaches that enforce constraints through explicit mathematical programming, the proposed RL framework learns to satisfy operational requirements implicitly through reward-based feedback during training.

## 4. Approach

The complete framework of the proposed approach is illustrated in Fig. 2. The framework comprises two main mechanisms: an RL-based optimization scheme and a tiled Voronoi partitioning module. This section first introduces the RL environment setup, including the definitions of the state, action, and observation spaces. It then presents the actor-critic learning architecture and provides pseudo-code describing the operational workflow. Finally, the tiled Voronoi partitioning used to generate sector boundaries is explained.

### 4.1. Environment setup

The RL environment models the interaction between air traffic, sector geometry, and ATCo workload. Since sector boundaries should be adjusted continuously in space, the environment is designed with continuous state and action spaces.

#### 4.1.1. Action space

The action space defines the allowable adjustments to sector centroids (Towers et al., 2024). In this study, the action at time step $t$ is defined as:

$$a_t = \left(A_i\right)_{i=1}^{N}, \qquad A_i = \left(a_{i,j}\right)_{j=1}^{D}, \qquad a_{i,j} \in [-1, 1] \tag{8}$$

where $N$ is the number of sectors and $D$ is the dimensionality of each sector's movement vector. In this study, $D = 3$ corresponds to adjustments in longitude, latitude, and altitude. Each component $a_{i,j}$ is bounded within $[-1, 1]$ to constrain the magnitude of allowable boundary adjustments.

#### 4.1.2. State space

The state contains all information required by the environment to evaluate an action and compute the resulting reward. After an action $a_t$ is applied at time $t$, the updated state is:

$$s_t = \left(\mathcal{C}_t,\ \mathcal{B}_t,\ \mathcal{F}_t,\ \mathcal{W}_t\right) \tag{9}$$

where the components are defined as follows.

The set of sector centroids $\mathcal{C}_t$ is defined as:

$$\mathcal{C}_t = \left(\mathbf{c}_t^i\right)_{i=1}^N, \qquad \mathbf{c}_t^i \in \mathbb{R}^2 \tag{10}$$

where each $\mathbf{c}_t^i$ lies within the projected UK airspace $\mathbb{R}^2$ and determines the geometric seeds $\mathcal{C}_t$ used in Voronoi-based partitioning.

The set of corresponding sector boundaries $\mathcal{B}_t$ is denoted by:

$$\mathcal{B}_t = \left(\mathbf{b}_t^i\right)_{i=1}^N, \qquad \mathbf{b}_t^i = \left(\mathbf{V}^{(i)}(\mathcal{C}_t)\right), \tag{11}$$

where each $\mathbf{b}_t^i$ is the boundary generated from the Voronoi diagram $\mathbf{V}^{(i)}$ built using all centroids $\mathcal{C}_t$.

Flight information $\mathcal{F}_t$ at time $t$ is defined as:

$$\mathcal{F}_t = \left(\tau_t^{f_p}\right)_{p=1}^{n_t}, \qquad t \in T \tag{12}$$

where $\tau_t^{f_p}$ contains the trajectory information of flight $f_p$ active at time $t$, $n_t$ is the number of such flights, and $T$ is the time set.

The ATCo workload $\mathcal{W}_t$ for all sectors is defined as:

$$\mathcal{W}_t = \left(\mathbf{W}_t^i\right)_{i=1}^N, \qquad \mathbf{W}_t^i = \Lambda\left(\mathbf{b}_t^i,\ \mathcal{F}_t\right), \qquad t \in T \tag{13}$$

where $\Lambda$ is the ATCo workload model that computes the workload $\mathbf{W}_t^i$ of sector $i$ at time $t$ based on its boundary $\mathbf{b}_t^i$ and the full set of active flights $\mathcal{F}_t$.

#### 4.1.3. Observation

In this study, the agent receives only a partial observation of the full state. At time $t$, the observation is:

$$o_t = \{\psi_t^i \mid i \in \{1, \ldots, N\},\ t \in T\} \tag{14}$$

where each $\psi_t^i$ contains the features of sector $i$ relevant for policy learning. Unlike the full state $s_t$, the observation excludes detailed geometry and complete flight trajectories.

Each observable feature vector $\psi_t^i$ is:

$$\psi_t^i = \left[\widetilde{w}_t^i,\ \widetilde{\rho}_t^i,\ \mathbf{c}_t^i\right] \tag{15}$$

where $\widetilde{w}_t^i \in [0, 1]$ is the normalized ATCo workload for sector $i$, $\widetilde{\rho}_t^i \in [0, 1]$ is the normalized sector similarity, and $\mathbf{c}_t^i$ is the centroid position of sector $i$.

### 4.2. Reinforcement learning architecture

Building on the MDP formulation defined in Section 3, the actor and critic networks are parameterized as follows, where $\pi_\theta$ and $V_\phi$ are the learned approximations of the optimal policy $\pi^*$ and its corresponding value function, respectively. Both networks are implemented as multi-layer perceptrons using the Stable-Baselines3 library.

#### 4.2.1. Actor network

The actor network is parameterized by $\theta$ and maps the observation $o_t \in \mathcal{O}$ to a probability distribution over the action space $\mathcal{A}$, presented as $\pi_\theta : \mathcal{O} \to \mathcal{P}(\mathcal{A})$.

Since the action space is continuous, the actor outputs the mean $\mu_\theta(o_t)$ and standard deviation $\sigma_\theta(o_t)$ of a Gaussian distribution over actions, from which the action $a_t \in \mathcal{A}$ is sampled: $a_t \sim \mathcal{N}\left(\mu_\theta(o_t),\ \sigma_\theta(o_t)^2\right)$.

#### 4.2.2. Critic network

The critic network is parameterized by $\phi$—the learnable weights of the critic neural network, distinct from the actor parameters $\theta$—and estimates the value of the current observation, providing a baseline to reduce the variance of policy gradient updates.

For PPO and Advantage Actor-Critic (A2C), the critic estimates the state value function $V_\phi(o_t)$, which represents the expected discounted cumulative reward from observation $o_t$ under the current policy $\pi_\theta$:

$$V_\phi(o_t) = \mathbb{E}_{\pi_\theta}\left[\sum_{t'=t}^{\infty} \gamma^{t'-t}\, \mathcal{R}(s_{t'}, a_{t'}, s_{t'+1}) \mid o_t\right] \tag{16}$$

For SAC, the critic instead estimates the state-action value function $Q_\phi(o_t, a_t)$, which additionally conditions on the action taken:

$$Q_\phi(o_t, a_t) = \mathbb{E}_{\pi_\theta}\left[\sum_{t'=t}^{\infty} \gamma^{t'-t}\, \mathcal{R}(s_{t'}, a_{t'}, s_{t'+1}) \mid o_t, a_t\right] \tag{17}$$

#### 4.2.3. Policy update

The actor and critic networks are updated iteratively during training. The critic is updated by minimizing the mean squared error between the predicted value and the estimated return $\hat{R}_t = \sum_{t'=t}^{T_{\max}} \gamma^{t'-t} \mathcal{R}(s_{t'}, a_{t'}, s_{t'+1})$, where $T_{\max}$ is the finite horizon length. For PPO and A2C, the critic loss $\mathcal{L}(\phi)$ on the state value function $V_\phi(o_t)$ is:

$$\mathcal{L}(\phi) = \mathbb{E}\left[\left(V_\phi(o_t) - \hat{R}_t\right)^2\right] \tag{18}$$

For SAC, the critic instead minimizes the Bellman error on the state-action value function $Q_\phi(o_t, a_t)$:

$$\mathcal{L}(\phi) = \mathbb{E}\left[\left(Q_\phi(o_t, a_t) - \hat{R}_t\right)^2\right] \tag{19}$$

The actor is updated by maximizing the expected advantage $\hat{A}_t = \hat{R}_t - V_\phi(o_t)$ for PPO and A2C, which measures how much better action $a_t$ is compared to the average action under the current policy $\pi_\theta$. The specific policy gradient update rule differs across the three algorithms: PPO applies a clipped surrogate objective to constrain the magnitude of policy updates (Schulman et al., 2017), A2C applies a standard advantage-based policy gradient (Mnih et al., 2016), and SAC uses $Q_\phi(o_t, a_t)$ and additionally incorporates an entropy regularization term to encourage exploration (Haarnoja et al., 2018).

The network parameters are updated via gradient-based optimization. The critic parameters $\phi$ are updated by gradient descent on $\mathcal{L}(\phi)$: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}(\phi)$. For PPO and A2C, the actor parameters $\theta$ are updated by gradient ascent on the expected advantage: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \mathbb{E}\left[\hat{A}_t \log \pi_\theta(a_t \mid o_t)\right]$. For SAC, the actor is instead updated by maximizing the expected $Q$-value plus an entropy regularization term: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \mathbb{E}\left[Q_\phi(o_t, a_t) + \beta \mathcal{H}(\pi_\theta(\cdot \mid o_t))\right]$, where $\beta$ is the temperature parameter controlling the trade-off between reward maximization and entropy, and $\mathcal{H}(\pi_\theta(\cdot \mid o_t))$ is the entropy of the policy at observation $o_t$. In all cases, $\alpha_\phi$ and $\alpha_\theta$ are the learning rates for the critic and actor, respectively. Both gradients are computed via backpropagation through the respective networks.

Algorithm 1 summarizes the full training and deployment procedure of the proposed framework, which forms the basis of the experimental evaluation presented in the following section.

### 4.3. Periodic Voronoi diagram

A core step in the RL framework is converting centroid positions into full sector boundaries. While Voronoi diagrams are well-suited for this task, the standard Voronoi formulation assumes an infinite plane, producing incomplete polygons along real-world airspace borders.

To address this, we incorporate the tiled (periodic) Voronoi method of Signoriello (2017). By tiling eight copies of the original point set around the main region, the Voronoi diagram is computed over an expanded domain. Sectors lying entirely within the original airspace boundary become complete, as illustrated in Fig. 3. This method integrates naturally into our RL workflow.

However, periodic Voronoi alone does not guarantee a fixed number of sectors, as boundary interactions may merge or split cells unpredictably. To preserve the required sector count, a post-processing step merges adjacent small regions when necessary.

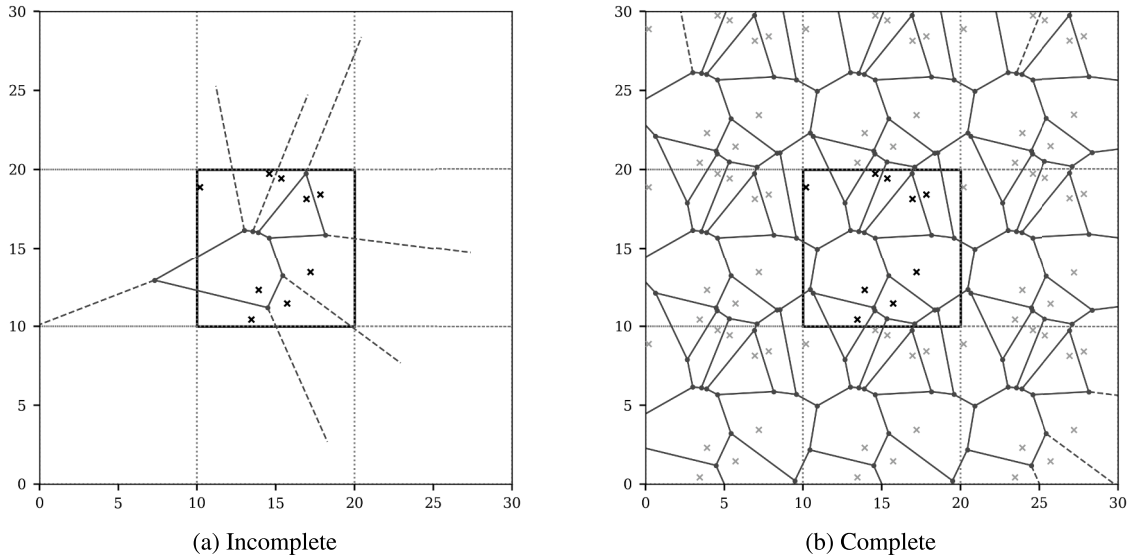



**Fig. 3.** Voronoi polygons at airspace borders: Fig. 3a presents incomplete boundaries without tiling; Fig. 3b shows complete boundaries with tiling.

**Algorithm 1:** Dynamic airspace sectorization solver using reinforcement learning.

TRAIN($\mathcal{F}, \mathcal{C}_0, \mathcal{B}_{\text{outer}}, \pi_0$)
**Input**: Flight trajectories $\mathcal{F}$, initial centroids $\mathcal{C}_0$, outer airspace boundary $\mathcal{B}_{\text{outer}}$, initial policy $\pi_0$;
**Output**: Optimized policy $\pi^*$;
**for** episode $k = 0$ **to** $K$ **do**
Initialize: $\mathcal{C} \leftarrow \mathcal{C}_0$, $\pi_\theta \leftarrow \pi_0$, $\mathcal{R}_{total} \leftarrow 0$;
**for** $t = 0$ **to** $T_{\max}$ **do**
Observe $o_t$ from $s_t$;
Select action $a_t \sim \pi_\theta(a_t|o_t)$;
Update centroids $\mathcal{C}_{t+1} \leftarrow \mathcal{C}_t + a_t$;
Generate sectors $\mathcal{B}_{t+1} \leftarrow$ TILEDVORONOI($\mathcal{C}_{t+1}, \mathcal{B}_{\text{outer}}$);
Compute workload $\mathcal{W}_{t+1} \leftarrow \Lambda(\mathcal{B}_{t+1}, \mathcal{F}_t)$;
Update state $s_{t+1} \leftarrow (\mathcal{C}_{t+1}, \mathcal{B}_{t+1}, \mathcal{F}_t, \mathcal{W}_{t+1})$;
Accumulate total reward $\mathcal{R}_{\text{total}} \leftarrow \mathcal{R}_{\text{total}} + \mathcal{R}(s_t, a_t, s_{t+1})$;
Update policy $\pi_\theta \leftarrow$ UPDATEPOLICY($\mathcal{R}_{total}, \pi_\theta$);
**end for**
**end for**
**return** Optimized policy $\pi^*$

PREDICT($\mathcal{F}', \pi^*, \mathcal{C}_0, \mathcal{B}_{\text{outer}}$)
**Input**: Predicted flights $\mathcal{F}'$, trained policy $\pi^*$, initial centroids $\mathcal{C}_0^*$, airspace boundary $\mathcal{B}_{\text{outer}}$;
**Output**: Optimal sectorization sequence $\{s_t^*\}_{t=0}^{T_{\max}}$;
**for** $t = 0$ **to** $T_{\max}$ **do**
Observe $o_t$ from $s_t$;
Select action $a_t^* \leftarrow \pi^*(o_t)$;
Update centroids $\mathcal{C}_{t+1}^* \leftarrow \mathcal{C}_t^* + a_t^*$;
Generate sectors $\mathcal{B}_{t+1}^* \leftarrow$ TILEDVORONOI($\mathcal{C}_{t+1}^*, \mathcal{B}_{\text{outer}}$);
Compute workload $\mathcal{W}_{t+1}^* \leftarrow \Lambda(\mathcal{B}_{t+1}^*, \mathcal{F}_t')$;
Store optimal state $s_{t+1}^* \leftarrow (\mathcal{C}_{t+1}^*, \mathcal{B}_{t+1}^*, \mathcal{F}_t', \mathcal{W}_{t+1}^*)$;
**end for**
**return** $\{s_t^*\}_{t=0}^{T_{\max}}$

## 5. Experiments

This section describes the experimental setup for training the RL agent on UK airspace data, including data preparation, airspace modeling, hyperparameter fine-tuning, and parallelism configuration.

### 5.1. Data preparation

The sectorization is initialized with the real-world UK airspace sectorization, which comprises 19 sectors. This fixed sector count is maintained throughout all experiments by a correction step—merging or splitting sectors as needed. In ATM practice, the number of open sectors is determined by available ATCo staffing and pre-tactical planning rather than being freely optimized (Tien, 2010). Treating the sector count as a given input is also consistent with the standard formulation of the airspace sectorization problem in the literature (Trandac et al., 2005). In this study, the sector count is an explicit, controllable input parameter—the solver can therefore be straightforwardly applied to any sector count, preserving its scalability across different ANSPs.

All flight data used in the case study are historical ADS-B data obtained from the OpenSky Network (Schäfer et al., 2014), which collects 1090 MHz Mode-S transmissions from publicly accessible receivers.

From this database, we extract over 200,000 flight trajectories within UK airspace spanning three representative weeks for training: 10–17 January 2023, 10–17 April 2023, and 10–17 August 2023. These weeks were selected from similar periods across different seasons to represent cyclical yet distinct traffic patterns, enabling more robust training. For testing, a later week of historical traffic during 17–23 November 2023 is used as "predicted" traffic to test the proposed framework, since real-world future flight plans are not accessible.

In this study, only flights operating within the en-route upper airspace band (FL 200–350) are considered, covering the primary cruising altitudes for commercial en-route traffic in UK airspace. For each flight, the extracted ADS-B data includes latitude, longitude, altitude, heading, ground speed, and timestamps sampled at a 60-second resolution. Each trajectory is represented as:

$$\mathbf{T}^{(e)} = \left(\boldsymbol{\tau}^{(e)},\ \mathbf{x}^{(e)},\ \mathbf{y}^{(e)},\ \mathbf{z}^{(e)},\ \mathbf{h}^{(e)},\ \mathbf{v}^{(e)}\right) \tag{20}$$

**Table 1**
Environmental parameter bounds used in the case study.

| Parameters | Values | Parameters | Values |
|---|---|---|---|
| $lat_{\min}$ | 49.48° | $lon_{\min}$ | −5.50° |
| $lat_{\max}$ | 61.00° | $lon_{\max}$ | 4.33° |
| $w_1$ | 0.4 | $w_2$ | 0.3 |
| $w_3$ | 0.2 | $w_4$ | 0.1 |

**Table 2**
Optimized hyperparameter settings.

| Agent | Number of steps per update $n_{step}$ | Batch size $b$ | Learning rate $l$ | Discount factor $\gamma$ | Clip range $\epsilon$ |
|---|---|---|---|---|---|
| PPO | 16 | 288 | 0.001 | 0.99 | 0.1 |
| A2C | 16 | N/A | 0.001 | 0.99 | N/A |
| SAC | 128 | 288 | 0.0001 | 0.99 | N/A |

where $\boldsymbol{\tau}^{(e)} = (\tau_1, \tau_2, \ldots, \tau_{L_e})$ denotes the timestamps, $\mathbf{x}^{(e)} = (x_1, x_2, \ldots, x_{L_e})$ the sequence of latitudes, $\mathbf{y}^{(e)} = (y_1, y_2, \ldots, y_{L_e})$ the longitudes, $\mathbf{z}^{(e)} = (z_1, z_2, \ldots, z_{L_e})$ the altitudes, $\mathbf{v}^{(e)} = (v_1, v_2, \ldots, v_{L_e})$ the ground speeds, and $\mathbf{h}^{(e)} = (h_1, h_2, \ldots, h_{L_e})$ the headings. The value $L_e$ denotes the number of vertices in trajectories.

Based on the actual UK airspace, Table 1 lists key environmental parameter values set in the model. Although Voronoi partitioning is performed within a simplified polygon at the beginning, the diagram will be combined with the actual airspace borders once it has been partitioned.

Prioritizing aircraft proximity and sector-crossing workload components, we assign the weight vector: $\boldsymbol{w} = [w_1, w_2, w_3, w_4] = [0.4, 0.3, 0.2, 0.1]$.

The first two workload terms, $\mathcal{W}_1$ and $\mathcal{W}_2$, typically range from thousands to tens of thousands, whereas $\mathcal{W}_3$ and $\mathcal{W}_4$ lie within $[0, 1]$. To ensure balanced optimization across all reward components, $\mathcal{W}_1$ and $\mathcal{W}_2$ are divided by $10^4$. Let the scaling matrix be: $\mathrm{diag}\left(1/10^4,\ 1/10^4,\ 1,\ 1\right)$.

### 5.2. Hyperparameter fine-tuning

Hyperparameters are configuration variables that control the learning process. Common hyperparameters include the learning rate $l$, discount factor $\gamma$, batch size $b$, number of steps per update $n_{step}$, and clip range $\epsilon$. Specifically, $l$ controls the update step size; $\gamma$ determines long-term reward contributions; $b$ is the number of samples per training batch; $n_{step}$ is the number of steps collected before each policy update. Notably, $b$ must be a multiple of $n_{step} \times n_{env}$. Here, $n_{env}$ represents the number of parallel environments running simultaneously.

To identify optimal hyperparameter combinations, we performed fine-tuning and pre-training using Bayesian optimization. Unlike grid search (exhaustive) or randomized search (uninformed sampling), Bayesian optimization iteratively selects hyperparameters based on prior results, enabling faster convergence to optimal values. The tuned hyperparameters for different RL agents are listed in Table 2.

### 5.3. Parallel processing

To accelerate training, vectorized environments are established so that the agent can interact with multiple independent environments simultaneously, accelerating learning through parallel experience collection. Action $\mathcal{A}$ and state $\mathcal{S}$ spaces are also vectorized, recording transitions from every parallel environment.

Since training in our case is primarily CPU-bound, we executed experiments on a Linux machine equipped with an AMD EPYC 7443 processor (24 physical cores, 48 logical threads) and two NVIDIA RTX A5000 GPUs (24 GB VRAM each). The system is well-suited for parallel workloads; 24 logical threads were utilized concurrently. This parallelism largely improves the convergence speed compared with the non-vectorized setup. The learning performance of the agent trained by the PPO, A2C, and SAC algorithms is compared in Section 6.

## 6. Results and discussion

This section presents the training convergence and testing performance of the three actor-critic algorithms (PPO, A2C, and SAC). We compare their performance in terms of workload reduction, sector dissimilarity, and computational efficiency against a state-of-the-art GA-based benchmark.

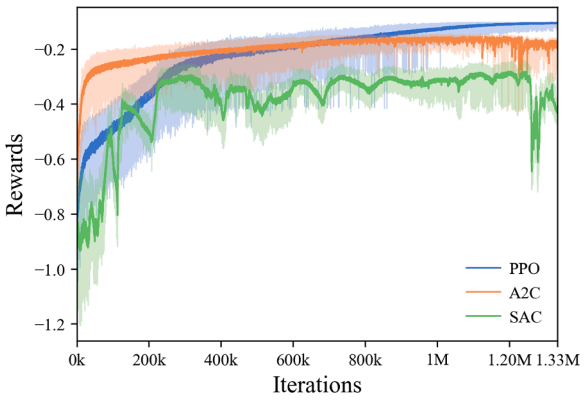



**Fig. 4.** Comparison of training performance across PPO, A2C, and SAC algorithms.

### 6.1. *Training performance*

During both training and testing, the proposed solver performs re-sectorization every four hours until the end of the time window. One training episode includes one week of traffic and therefore requires 42 sequential re-sectorizations.

Fig. 4 shows the convergence behavior of the three algorithms during training. The PPO-trained agent demonstrates the most stable learning trajectory and achieves the highest cumulative reward (lowest penalty) after 1.3 million training timesteps. The A2C-trained agent exhibits faster initial convergence but experiences performance degradation in later training stages, ultimately achieving a lower final reward than PPO. Nevertheless, A2C's rapid early convergence makes it suitable for applications with limited training budgets.

In contrast, the SAC-trained agent exhibits unstable learning dynamics throughout the training process. Although SAC briefly achieves reward levels comparable to PPO, it experiences persistent fluctuations and ultimately suffers significant performance degradation in the final training phase. These results demonstrate that PPO is the most suitable algorithm for the DAS problem, providing both superior convergence stability and final performance. All three algorithms were trained with optimized hyperparameters obtained through Bayesian optimization, ensuring that the observed performance differences reflect algorithmic characteristics rather than suboptimal parameter selection.

Based on these training results, the PPO-trained agent is selected for testing, as it achieves the highest final reward with the most stable convergence behavior.

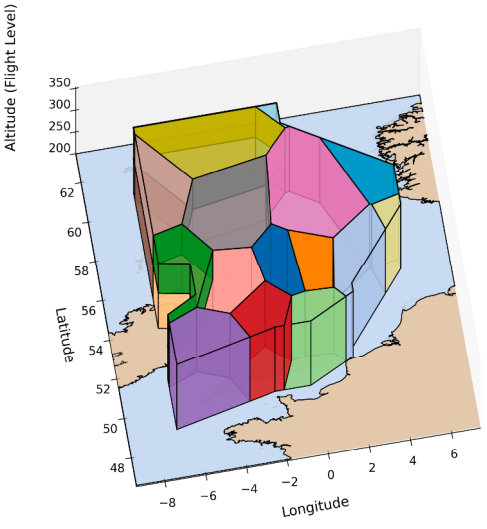
### 6.2. *Implementation outputs*

Historical flight patterns in UK airspace during 17–23 November 2023 are used for testing, as this constitutes a busy week in the autumn to winter period. The one-week period is divided into 42 intervals (each lasting four hours), with a re-sectorization performed at the start of each.

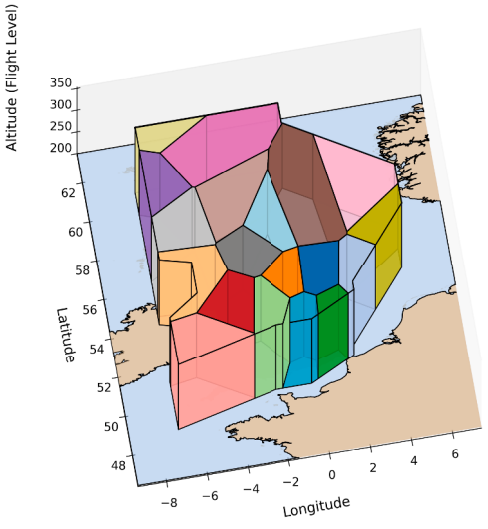
Fig. 5a visualizes the elevated view of the first sectorization optimized for the first four-hour interval, in comparison to the original static sectorization (Fig. 5b) and that produced by a GA-based benchmark for the first interval (Fig. 5c). As observed, our solver initially generated fifteen horizontal sectors, and four of these sectors were further subdivided vertically due to higher traffic density in those regions.

To evaluate the performance and statistical robustness of the PPO-trained solver against varying traffic demand, we tested it under ±20% demand multipliers across all six intervals of a representative operational day (22 November 2023). Table 3 summarizes the average responses of the PPO solver in terms of total reward and runtime under different demand scales. Positive values indicate relative performance improvements, while negative values indicate degradation relative to the 1.0× baseline. As observed, both total reward and runtime vary smoothly with demand. Under reduced demand (80–90%), the solver achieves a 4–9% reward improvement and an 11–19% reduction in runtime, indicating more efficient operation when traffic is lighter. Increasing demand to 120% results in a modest 3–5% deterioration in reward performance, as expected with higher traffic complexity. The narrow 95% confidence intervals across all cases confirm stable solver behavior even under scaled demand conditions. Notably, runtime does not show a statistically significant increase with higher demand. The absence of a significant runtime trend ($p = .074$) suggests that the computational effort is not proportional to traffic intensity. This confirms the scalability of the PPO solver: its computational complexity does not grow linearly with the number of flights, enabling stable computation and predictable performance even in high-demand scenarios. Such behavior is essential for real-time DAS applications and supports future integration into DCB processes.

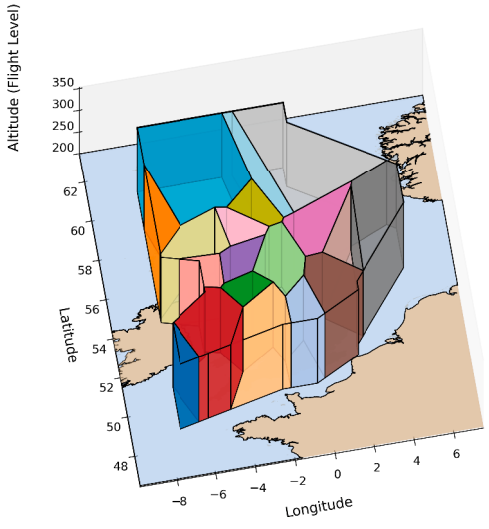
To further evaluate the solver's robustness under extreme traffic conditions, a stress test was conducted at 150% demand. The solver exhibits only a modest performance degradation of 3.5% in total workload, and the change in runtime is negligible. These results demonstrate that even under highly congested scenarios, the PPO solver maintains performance while preserving computational efficiency, demonstrating strong scalability for real-time deployment.




(a) PPO-generated sectorization




(b) Real-world static baseline sectorization

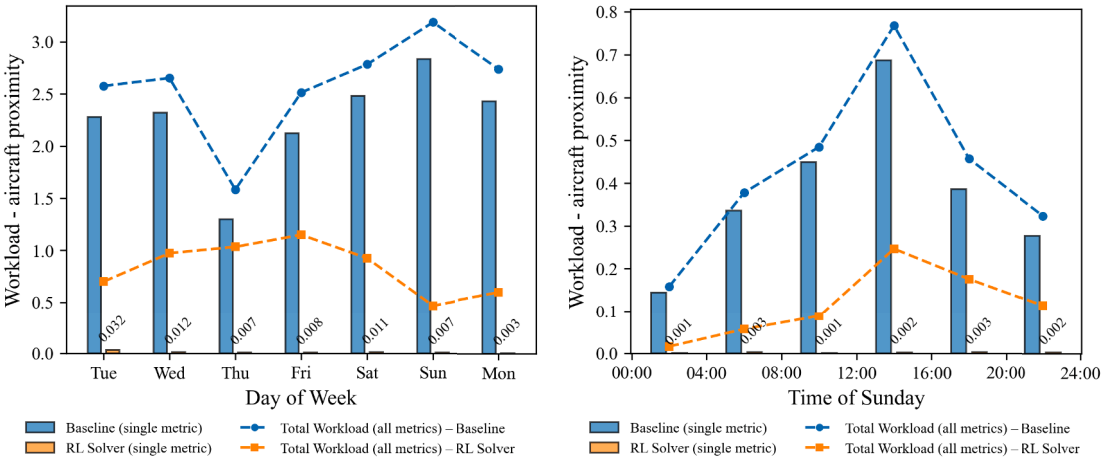



(c) GA-generated sectorization

**Fig. 5.** Elevated view of sectorization solutions. Fig. 5a shows the sectorization optimized for time interval 1 by the PPO solver. Fig. 5b shows the original static UK airspace sectorization. Fig. 5c shows the sectorization optimized for time interval 1 by the GA-based benchmark.

**Table 3**
Sensitivity analysis of PPO solver performance under scaled traffic demand. Positive values of mean Δ indicate that the PPO solver outperforms the baseline.

| Demand Scale | Mean Total Reward (%) | 95% CI | Mean Runtime (%) | 95% CI |
|---|---|---|---|---|
| 0.8× | +9.0 | [5.7, 12.3] | −18.2 | [−19.7, −16.7] |
| 0.9× | +4.1 | [1.6, 6.7] | −10.6 | [−12.8, −8.5] |
| 1.0× (default) | 0.0 | [–, –] | 0.0 | [–, –] |
| 1.1× | −2.7 | [−4.9, −0.4] | +1.8 | [−0.4, +4.1] |
| 1.2× | −5.2 | [−7.3, −3.1] | +1.1 | [−1.4, +3.7] |
| 1.5× (stress test) | −3.5 | [−6.3, −0.7] | +0.0 | [−2.4, +2.4] |
| *p*-value (overall) | < 0.001 | N/A | 0.074 | N/A |




(a) Daily aircraft proximity-related workload (b) Hourly aircraft proximity-related workload

**Fig. 6.** Aircraft proximity-related workload of the PPO-trained solver compared with the original UK airspace sectorization. Left: daily aggregated workload over 17–23 November 2023. Right: workload at four-hour intervals on 22 November 2023. Dotted lines indicate total ATCo workload and bars represent specific workload components, with the PPO-trained solver and the original sectorization distinguished by color.

### *6.3. Baseline comparison*

We compare the performance of the re-sectorizations produced by the PPO-trained solver with the original UK airspace sectorization on identical air traffic patterns at each interval during 17–23 November 2023. Figs. 6–8 show daily and hourly workload levels for each component across the full week and within a representative day, respectively. The dotted lines indicate the total ATCo workload, while the bars represent the individual workload components.

Across all seven days, the PPO-trained solver consistently reduces the total ATCo workload relative to the original static sectorization, with daily reductions ranging between 33% and 80%. This persistent gap across varying traffic conditions demonstrates the solver's generalization capability beyond the training period.

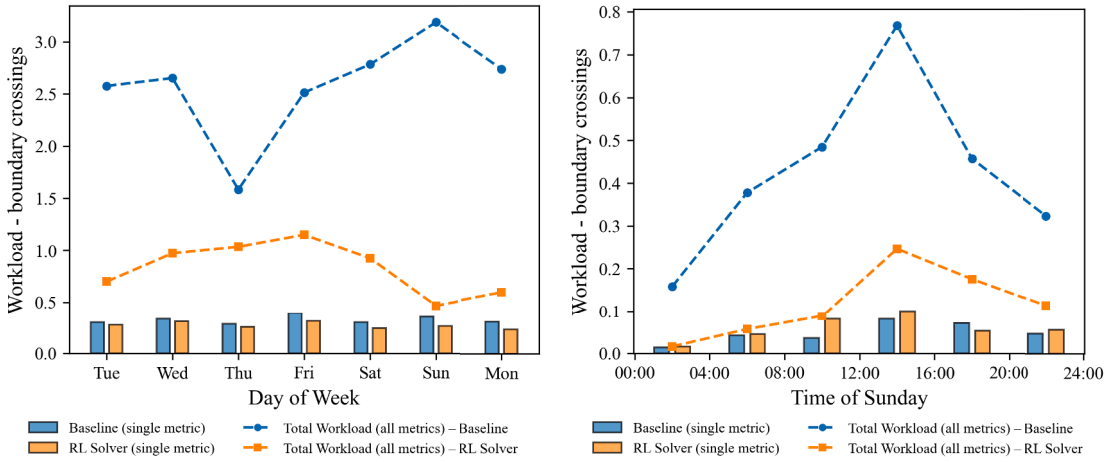
Fig. 6a shows the daily aircraft proximity-related workload across the testing week. This component is the dominant contributor to the total ATCo workload under the static baseline, with daily values ranging from approximately 1.3 on Thursday to 2.9 on Sunday, closely following the total workload trend. Under the PPO solver, proximity workload is reduced to near zero on all days, with a maximum of 0.032 on Tuesday, representing a reduction exceeding 98% on most days. This near-complete elimination of aircraft proximity occurring close to boundaries is the primary driver of the overall workload reduction observed throughout the week.

As for four-hour intervals on Sunday, 22 November 2023, the total workload (represented by dotted lines) shows a significant reduction under the PPO solver compared with the baseline. The baseline total workload rises from around 0.15 at 00:00–04:00 to a peak of approximately 0.75 at 12:00–16:00, before gradually declining toward 0.35 by 20:00–24:00. In contrast, the PPO solver maintains total workload below 0.25 throughout the day, achieving reductions of 60–85% across all intervals. Fig. 6b shows aircraft proximity-related workload across the intervals on Sunday. This workload component is still the largest contributor to total ATCo workload under both the baseline and optimized configurations. Under the static baseline, proximity workload ranges from 0.13 at 00:00–04:00 to approximately 0.70 during peak midday periods, whereas the PPO solver reduces it to below 0.003 across all intervals—a reduction exceeding 95%. This substantial decrease demonstrates that the PPO solver effectively minimizes aircraft interactions along sector boundaries throughout the day.

Fig. 7a shows the daily sector crossing-related workload, which corresponds to flight handovers between adjacent ATCos. The baseline crossing workload is modest relative to the proximity component, ranging from approximately 0.2 to 0.4 across the week. The PPO solver produces comparable but slightly lower crossing workload. This modest improvement reflects the same trade-off observed in the hourly workload comparison: by reconfiguring sector boundaries to minimize proximity events, the solver is less strict on handovers.

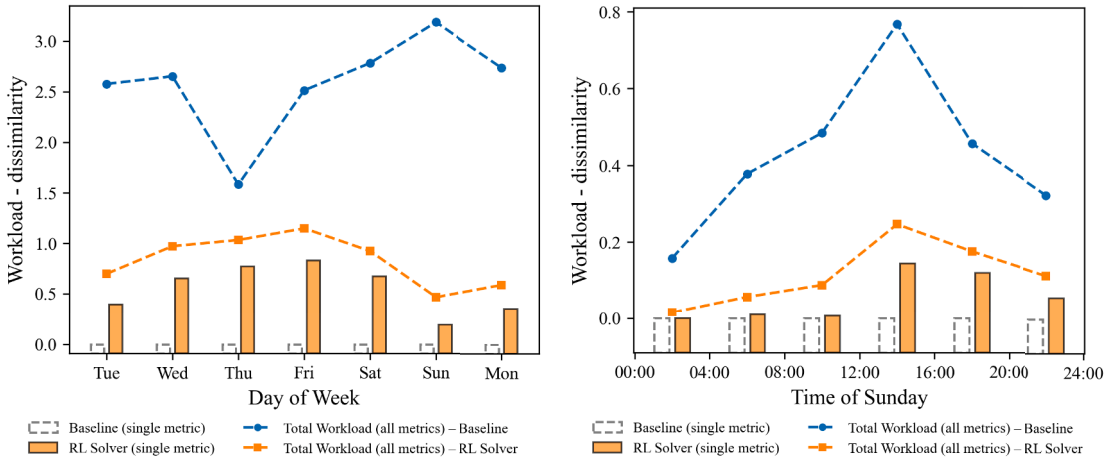
Fig. 7b shows the hourly sector crossing-related workload. The static baseline exhibits crossing workload ranging from 0.015 (during overnight hours, 00:00–04:00) to 0.10 during peak traffic (12:00–16:00). The PPO solver produces slightly higher crossing workload across most intervals, with the exception of the fifth period (16:00–20:00). This modest increase represents a deliberate trade-off: by prioritizing proximity reduction, the solver occasionally creates additional sector boundaries that increase normal handovers but substantially reduce the more costly coordination to prevent conflicts.

Fig. 8a shows the daily sectorization change-related workload, which quantifies the adaptation burden introduced by boundary changes between consecutive intervals. As the static baseline never alters its sectorization, its dissimilarity workload is zero throughout. The PPO solver exhibits considerable day-to-day variation in this component, ranging from 0.195 on Sunday to 0.836 on Friday.




(a) Daily sector crossing-related workload (b) Hourly sector crossing-related workload

**Fig. 7.** Sector crossing-related workload of the PPO-trained solver compared with the original UK airspace sectorization. Left: daily aggregated workload over 17–23 November 2023. Right: workload at four-hour intervals on 22 November 2023. Dotted lines indicate total ATCo workload and bars represent specific workload components, with the PPO-trained solver and the original sectorization distinguished by color.




(a) Daily sectorization change-related workload (b) Hourly sectorization change-related workload

**Fig. 8.** Sectorization change-related workload of the PPO-trained solver compared with the original UK airspace sectorization. Left: daily aggregated workload over 17–23 November 2023. Right: workload at four-hour intervals on 22 November 2023. Dotted lines indicate total ATCo workload and bars represent specific workload components, with the PPO-trained solver and the original sectorization distinguished by color.

The elevated values on Thursday and Friday suggest that higher traffic complexity on those days requires more substantial boundary adjustments across intervals. Conversely, the low dissimilarity on Sunday indicates that the solver identifies a relatively stable sectorization well-suited to Sunday traffic patterns, resulting in fewer boundary changes throughout that day.

Fig. 8b shows the workload related to sectorization changes between consecutive re-sectorizations throughout the representative day. The static baseline maintains zero dissimilarity. The PPO solver exhibits low dissimilarity during low-traffic periods (00:00–12:00) but increases to 0.1–0.2 during high-traffic periods (12:00–20:00), indicating more substantial boundary adjustments to accommodate peak demand. This hourly pattern is consistent with the daily pattern, and confirms that the solver adaptively modifies sector configurations in response to changing traffic conditions.

A paired $t$-test compares the PPO-trained solver against the static baseline across the seven evaluation days, with each day's aggregated workload forming one paired observation. As shown in Table 4, the proposed solver achieves statistically significant reductions in total ATCo workload and in both proximity and crossing components. Total workload decreases by an average of 1.743 ($t = 6.84$, $p < .001$, $d = 2.59$), representing a large effect size. Proximity workload shows the largest individual reduction ($\Delta = 2.240$, $t = 12.49$, $p < .001$, $d = 4.72$). Crossing workload also decreases significantly ($\Delta = 0.055$, $t = 5.19$, $p = .002$, $d = 1.96$), in contrast to

**Table 4**
Paired *t*-test validation of PPO solver performance against the static baseline over seven evaluation days. Positive values of mean Δ indicate that the PPO solver outperforms the baseline.

| Workload | Mean Δ | SE | *t* | *p*-value | Cohen's *d* |
|---|---|---|---|---|---|
| Aircraft proximity-related workload | +2.240 | 0.179 | 12.49 | < 0.001 | 4.72 |
| Sector crossing-related workload | +0.055 | 0.011 | 5.19 | 0.002 | 1.96 |
| Sectorization change-related workload | −0.552 | 0.090 | −6.12 | < 0.001 | −2.31 |
| Total workload | +1.743 | 0.255 | 6.84 | < 0.001 | 2.59 |

the single-day analysis, where this difference is negligible. As expected, sectorization change-related workload increases significantly under the PPO solver ($\Delta = -0.552$, $t = -6.12$, $p < .001$, $d = -2.31$), reflecting the adaptation burden introduced by dynamic boundary adjustments.

Since the static baseline does not involve re-sectorization, it provides no basis for evaluating adaptation workload or computational efficiency. Therefore, we compare the PPO-trained solver against a benchmark to assess performance in terms of sectorization continuity and computational speed.

### 6.4. Benchmark comparison

The benchmark is a state-of-the-art GA-based solver adapted for UK airspace. To ensure a fair comparison, both solvers use the same workload-based objective function (Section 3.2) on identical flight trajectories. A key distinction is that the GA approach treats each time period independently and addresses sectorization similarity only through post-processing adjustments, as GA lacks native support for sequential decision making across time periods. In contrast, the RL framework directly optimizes temporal continuity during training. Algorithm 2 outlines the GA solver's procedure.

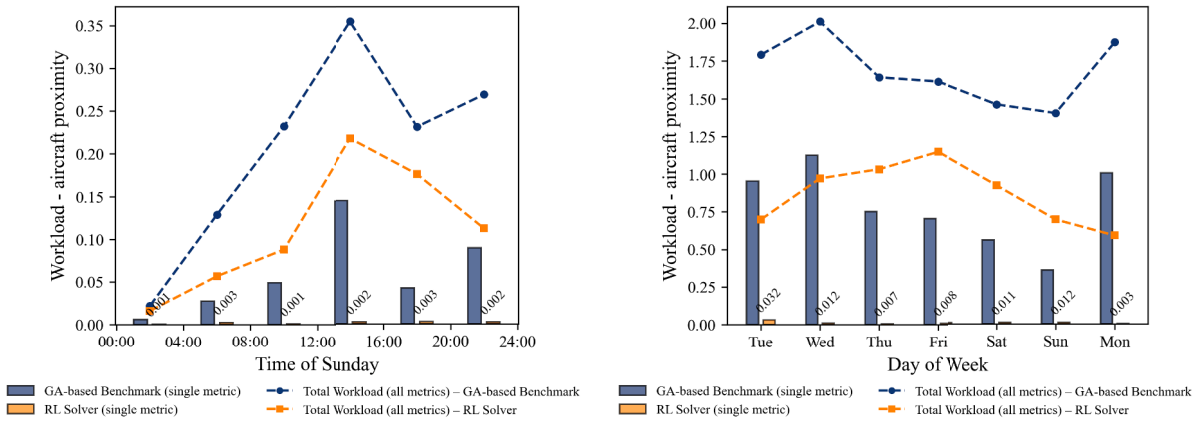
**Algorithm 2:** Dynamic airspace sectorization solver using genetic algorithm.

GA SOLVE($s_0, b, f', PopSize, MaxGen, p_c, p_m$)
**Input**: initial sectorization $s_0$, outer airspace borders $b$, predicted flights $f'$, population size $PopSize$, maximum generation $MaxGen$, crossover probability $p_c$, mutation probability $p_m$;
**Output**: Optimized Re-sectorization $s_i^*$;
**for** $i = 0$ **to** $I$ **do**
  Initialization: $p \leftarrow$ GENPOP($s_0, PopSize$);
  **for** $g = 1$ **to** $MaxGen$ **do**
    Parent set $P \leftarrow$ SELECT($p$);
    Offspring $O \leftarrow$ CROSSOVER($P, p_c$);
    Mutated offspring $O' \leftarrow$ MUTATE($O, p_m$);
    Fitness $F \leftarrow$ FITNESS($O', f'$);
    Selected survivor $s' \leftarrow$ SURVIVOR($O', f'$);
    **if** TERMINATECRITERION **then**;
      $G^* += s_i^*$;
      break;
    **end if**
  **end for**
  $i \leftarrow i + 1$;
**end for**
**return** $s_i^*$

The PPO solver substantially outperforms the GA approach across all time intervals, achieving a lower total ATCo workload throughout the day. During peak traffic periods (12:00–16:00) on Sunday, the GA solver's normalized workload reaches approximately 0.35, while the PPO solver maintains a workload below 0.20, representing an average reduction of 50.59% relative to the GA baseline.

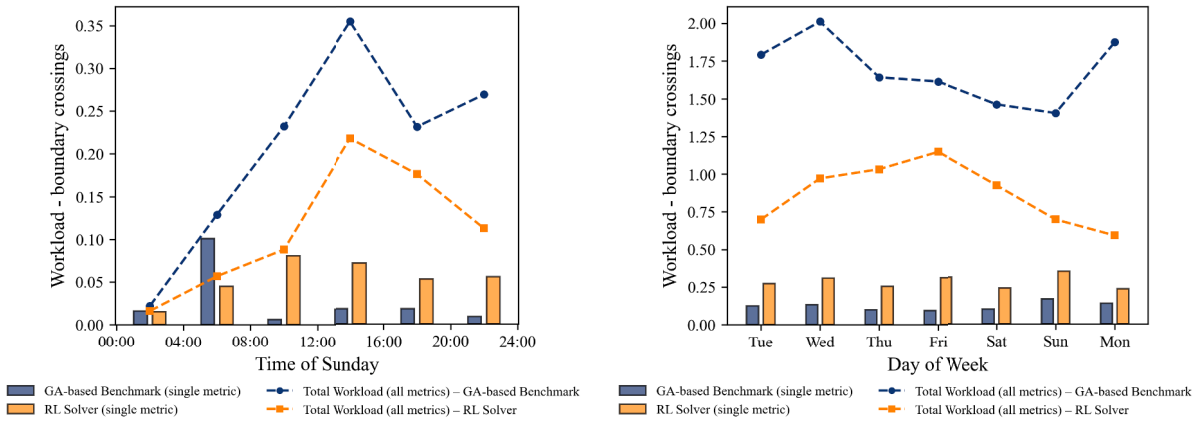
Fig. 9a shows the hourly aircraft proximity-related workload comparison between the two solvers. The PPO solver remains near-zero across all intervals (daily values ranging from 0.003 to 0.032), while the GA baseline sustains substantially higher proximity workload throughout the day. This near-complete elimination of proximity workload is the primary driver of the overall workload reduction achieved by the PPO solver.

Fig. 9b extends the comparison to daily aggregated proximity workload over the full evaluation week (17–23 November 2023). The GA baseline sustains substantially higher proximity workload every day, peaking on Wednesday (1.125) and remaining elevated on Monday (1.006) when traffic demand is highest. On average, the GA solver's proximity workload is approximately 65 times that




(a) Hourly aircraft proximity-related workload (b) Daily aircraft proximity-related workload

**Fig. 9.** Aircraft proximity-related workload comparison between the PPO-trained solver and the GA-based benchmark. Left: workload at four-hour intervals on 22 November 2023. Right: daily aggregated workload over the week of 17–23 November 2023. In all subplots, dotted lines indicate total ATCo workload and bars represent specific workload components, with the PPO-trained solver and the GA-based benchmark distinguished by colour.




(a) Hourly sector crossing-related workload (b) Daily sector crossing-related workload

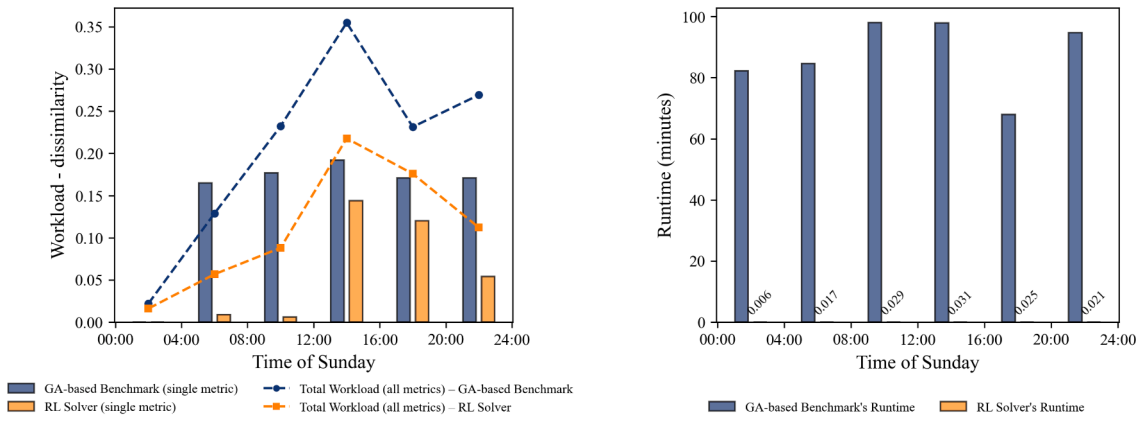
**Fig. 10.** Sector crossing-related workload comparison between the PPO-trained solver and the GA-based benchmark. Left: workload at four-hour intervals on 22 November 2023. Right: daily aggregated workload over the week of 17–23 November 2023. In all subplots, dotted lines indicate total ATCo workload and bars represent specific workload components, with the PPO-trained solver and the GA-based benchmark distinguished by colour.

of the PPO solver across the seven days, confirming that the PPO solver's advantage in this component is robust across varying daily traffic conditions.

Fig. 10a compares the sector boundary crossing workload between the two solvers. The PPO solver maintains a crossing workload between 0 and 0.05 during the first two intervals (00:00–08:00), while the GA solver achieves slightly lower values (0–0.025) from 08:00 onward, representing a 15–40% reduction in this specific metric.

Fig. 10b shows the daily crossing workload over the full evaluation week. The GA solver consistently achieves lower daily values (daily average: 0.122) compared to the PPO solver (daily average: 0.282), which is consistent with the hourly results reported above. Despite this disadvantage in the crossing component, the PPO solver's near-elimination of proximity workload yields a substantially lower total ATCo workload across all seven days, with an average reduction of 50.59% relative to the GA baseline (ranging from 29% on Friday to 69% on Monday).

Fig. 11a shows the hourly sectorization change-related workload under both solvers. The GA solver exhibits consistently high dissimilarity (0.15–0.20 across all intervals), indicating complex boundary changes between consecutive time periods. In contrast, the PPO solver maintains significantly lower dissimilarity, ranging from near-zero during low-traffic periods to 0.13 during peaks—an average reduction of approximately 60%. This lower dissimilarity reduces the ATCo adaptation burden, making this approach more




(a) Hourly sectorization change-related workload

(b) Hourly solver runtime

**Fig. 11.** Sectorization change-related workload and solver runtime comparison between the PPO-trained solver and the GA-based benchmark at four-hour intervals on 22 November 2023. Dotted lines indicate total ATCo workload and bars represent specific workload components, with the PPO-trained solver and the GA-based benchmark distinguished by colour.

**Table 5**
Paired $t$-test validation of PPO solver improvement over GA benchmark. Positive values indicate that the PPO solver outperforms GA.

| Metrics | Mean Δ | $SE$ | $t$ | $p$-value | Cohen's $d$ |
|---|---|---|---|---|---|
| Aircraft proximity-related workload | +0.768 | 0.101 | 7.59 | < 0.001 | 2.87 |
| Sector crossing-related workload | −0.160 | 0.014 | −11.14 | < 0.001 | −4.21 |
| Sector change-related workload | +0.212 | 0.078 | 2.73 | 0.034 | 1.03 |
| Total ATCo workload | +0.820 | 0.120 | 6.85 | < 0.001 | 2.59 |
| Hourly runtime (seconds) | +5234.4 | 313.8 | 16.68 | < 0.00001 | 6.67 |

operationally feasible for real-world applications. Since the GA-based solver cannot optimize changes between re-sectorizations, the daily aggregated dissimilarity-related workload is not specifically displayed.

Fig. 11b compares the computational efficiency between the PPO solver and the benchmark. The PPO solver requires only an average of 1.31 seconds per re-sectorization decision, while the GA benchmark requires an average of 82.22 minutes (up to 100 minutes for complex intervals). This represents a 99.98% reduction in computation time. The near-instantaneous runtime of the PPO solver enables real-time implementation, whereas the GA approach remains computationally costly for time-critical applications. Since the GA-based solver processes each interval independently, the daily aggregated runtime is simply the sum of per-interval runtimes, which are already reported; given the consistently large runtime advantage of the PPO solver across all intervals, a separate daily runtime comparison offers no additional insight and is therefore not displayed.

The comparison confirms the improvement of the proposed approach in reducing ATCo workload by generating optimized and consistent airspace re-sectorization. Paired $t$-tests compare the PPO and GA solvers across time intervals (Table 5). The PPO solver significantly outperforms the GA baseline on aircraft proximity workload ($t = 7.59$, $p < .001$, Cohen's $d = 2.87$), sectorization dissimilarity ($t = 2.73$, $p = .034$, $d = 1.03$), total ATCo workload ($t = 6.85$, $p < .001$, $d = 2.59$), and computation time ($t = 16.68$, $p < .001$, $d = 6.67$). The GA solver achieves lower sector crossing workload ($t = -11.14$, $p < .001$, $d = -4.21$), consistent with the results reported above. Overall, these results demonstrate that the PPO solver delivers consistent and reliable performance advantages over the GA baseline across the key operational metrics.

An additional operational advantage is that the PPO solver, once trained, can operate independently without requiring future flight trajectory predictions in advance, whereas the GA approach must re-optimize using future traffic data for each re-sectorization. These combined advantages—superior solution quality, real-time performance, and operational simplicity—make the RL framework a practical solution for intelligent ATM systems.

## 7. Conclusion and future work

This paper has presented a novel RL-based framework that is universally applicable to DAS problems. In this framework, an RL agent is trained to optimize the DAS policy by exploring and exploiting different DAS solutions for historical scenarios. Equipped with actor-critic RL and periodic Voronoi partitioning, the framework can directly adapt existing sectorizations to dynamic traffic conditions, reflecting the practical re-sectorization process. The use of periodic Voronoi diagrams also mitigates the inherent limitation of conventional Voronoi partitioning by ensuring complete sector boundaries at airspace borders.

The proposed solver was trained on three weeks of historical flight data from UK airspace and evaluated on traffic over a full week in November 2023. Once training is completed, the solver re-sectorizes the airspace in an average of 1.31 seconds. Compared with a GA-based benchmark, the proposed solver achieved a 50.59% reduction in ATCo workload and a 99.98% reduction in computation time. The performance gap is even more pronounced when compared with the original static UK sectorization. Paired t-tests confirm that these gains are statistically significant ($p < .001$ for total workload and $p < .00001$ for runtime, relative to both the static baseline and the GA benchmark), and sensitivity analysis confirms stable solver behavior across demand scales from 0.8 to 1.5.

A key practical consideration is how this framework would be embedded within operational ATM systems, such as those used by National Air Traffic Services in the UK or managed at the network level by EUROCONTROL. In current operations, sectorization decisions are made pre-tactically during the air traffic flow and capacity management planning cycle and are subject to approval before being communicated to sector teams. The proposed RL-based solver is designed to function as a decision-support tool within the pre-tactical planning phase, generating candidate sectorization sequences from predicted traffic demand that supervisors then review and approve. The near-instantaneous inference time makes it equally suited to the tactical phase, where it could be triggered reactively by demand forecast updates or sector workload alerts. In either mode, the framework does not require modification to sector ownership structures, communication protocols, or flight data processing pipelines, as it operates directly on trajectory data in formats consistent with existing ADS-B and flight plan feeds.

The present study lays the groundwork for deployment; the following implementation steps remain for future work. First, integration with live flight plan data would be necessary for operational use, as real-time DCB depends on predicted rather than observed traffic. Second, the solver's outputs would need to be validated against current ATM safety and human factors standards, including minimum notification times for sector reconfiguration and maximum permitted boundary change magnitudes per interval. Third, while the solver minimizes the mathematical workload burden of re-sectorization, ATCo acceptance of algorithmically generated sector boundaries will depend on the interpretability and consistency of the proposed configurations; therefore, human-in-the-loop is necessary to bridge this gap. Future work should therefore involve simulation exercises and potentially live trials with experienced ATCos to evaluate operational acceptance alongside computational performance.

Although this study was conducted in the UK en-route upper airspace, the framework is designed to be airspace-agnostic in its architecture, and its validation in a high-complexity environment supports its applicability to other regions. The core components make no assumptions specific to UK airspace geometry, traffic density, or sector count. Adapting the framework to a different airspace requires three inputs: historical ADS-B or radar trajectory data for training, a definition of the outer airspace boundary, and a target sector count consistent with local ANSP staffing norms. The hyperparameter tuning conducted via Bayesian optimization in this study needs to be repeated for substantially different traffic densities or airspace geometries; though the tuning procedure itself is fully automated and does not require domain-specific adjustment.

The framework is well-suited to dense, complex airspaces where the limitations of static sectorization and the computational cost of GA-based dynamic solvers are most acute. Conversely, in airspaces with low traffic density or where sector reconfiguration is infrequent due to operational policy, the performance benefits relative to a static baseline would likely be more modest. The framework's scalability has been validated here for 19 sectors; extending it to airspaces requiring substantially more sectors would increase the dimensionality of the action space and may require architectural modifications to the actor network, such as factorized or hierarchical action representations. This constitutes an important direction for future research.

In summary, the proposed framework not only provides an effective DAS solution but also demonstrates strong potential for integration within broader DCB applications, owing to its computational efficiency, operational compatibility, and structural transferability. Future work will extend the framework by coupling the RL-based sectorization module with trajectory adjustment components to support fully integrated, real-time DCB systems and by conducting human-in-the-loop evaluations to assess operational readiness.

## CRediT authorship contribution statement

**Wenxuan Wang:** Writing – original draft, Visualization, Validation, Methodology, Investigation; **Arnab Majumdar:** Supervision; **Washington Ochieng:** Supervision; **Jose Escribano:** Writing – review & editing, Supervision, Conceptualization.

## Data availability

The code and data used in this study are publicly available at https://github.com/Haileywww/RL_DAS.

## References

Bolić, T., Ravenhill, P., 2021. Sesar: the past, present, and future of European air traffic management research. Engineering 7 (4), 448–451.

Bolić, T., Castelli, L., Corolli, L., Rigonat, D., 2017. Reducing ATFM delays through strategic flight planning. Transport. Res. Part E 98, 42–59. https://doi.org/10.1016/j.tre.2016.12.001

Chaloulos, G., Crück, E., Lygeros, J., 2010. A simulation based study of subliminal control for air traffic management. Transp. Res. Part C 18 (6), 963–974.

Chen, Y., Bi, H., Zhang, D., Song, Z., 2013. Dynamic airspace sectorization via improved genetic algorithm. J. Mod. Transp. 21, 117–124.

Chen, Y., Dalmau, R., Alam, S., 2025. Tactical demand and capacity balancing using incremental search in spatio-temporal graphs with flight uncertainty. Transport. Res. Part C. 181, 105382. https://doi.org/10.1016/j.trc.2025.105382

Chen, Y., Xu, Y., Hu, M., 2023. General multi-agent reinforcement learning integrating heuristic-based delay priority strategy for demand and capacity balancing. Transport. Res. Part C. 153, 104218. https://doi.org/10.1016/j.trc.2023.104218

De Berg, M., Cheong, O., Van Kreveld, M., Overmars, M., 2008. Computational Geometry: Algorithms and Applications. Springer.

Delahaye, D., Schoenauer, M., Alliot, J.-M., 1998. Airspace sectoring by evolutionary computation. In: 1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence (Cat. No. 98TH8360). IEEE, pp. 218–223.

Delgado, L., Gurtner, G., Bolić, T., Castelli, L., 2021. Estimating economic severity of air traffic flow management regulations. Transport. Res. Part C. 125, 103054.
Dhief, I., Alam, S., Lilith, N., Mean, C.C., 2022. A machine learned go-around prediction model using pilot-in-the-loop simulations. Transport. Res. Part C. 140, 103704. https://doi.org/10.1016/j.trc.2022.103704
Djokic, J., Lorenz, B., Fricke, H., 2010. Air traffic control complexity as workload driver. Transport. Res. Part C. 18 (6), 930–936. Special issue on Transportation Simulation Advances in Air Transportation Research. https://doi.org/10.1016/j.trc.2010.03.005
Gerdes, I., Temme, A., Schultz, M., 2016. Dynamic airspace sectorization using controller task load. 6th SESAR Innov. Days , 1–10.
Gerdes, I., Temme, A., Schultz, M., 2018. Dynamic airspace sectorisation for flight-centric operations. Transport. Res. Part C. 95, 460–480.
Haarnoja, T., Zhou, A., Abbeel, P., Levine, S., 2018. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International Conference on Machine Learning. Pmlr, pp. 1861–1870.
Hind, H., El Omri, A., Abghour, N., Moussaid, K., Rida, M., 2018. Dynamic airspace configuration: review and open research issues. In: 2018 4th International Conference on Logistics Operations Management (GOL). IEEE, pp. 1–7.
Hu, Y., Du, Y., Li, S., 2025. Real-time coordination of human couriers and drones for on-demand food-delivery platforms: a multi-stage risk-aware multi-agent reinforcement learning framework. Transport. Res. Part C. 181, 105381. https://doi.org/10.1016/j.trc.2025.105381
Kopardekar, P., Bilimoria, K., Sridhar, B., 2007. Initial concepts for dynamic airspace configuration. In: 7th AIAA ATIO Conf, 2nd CEIAT Int'l Conf on Innov and Integr in Aero Sciences, 17th LTA Systems Tech Conf; Followed by 2nd TEOS Forum, p. 7763.
Kopardekar, P., Bilimoria, K.D., Sridhar, B., 2008. Airspace configuration concepts for the next generation air transportation system. Air Traffic Control Quart. 16 (4), 313–336.
Liu, Z., Xiao, G., Mao, J., 2025. An integrated multi-objective model for demand-capacity balancing and strategic de-confliction under autonomous aircraft flight. Transport. Res. Part C. 174, 105102. https://doi.org/10.1016/j.trc.2025.105102
Lui, G.N., Lulli, G., Lema-Esposto, M.F., Martinez, R.L., 2024. Airspace sector design: an optimization approach. SESAR Innov. Days .
Mannino, C., Nakkerud, A., Sartor, G., 2021. Air traffic flow management with layered workload constraints. Comput. Oper. Res. 127, 105159. https://doi.org/10.1016/j.cor.2020.105159
Martinez, S., Chatterji, G., Sun, D., Bayen, A., 2007. A weighted-graph approach for dynamic airspace configuration. In: Aiaa Guidance, Navigation and Control Conference and Exhibit, p. 6448.
Melgosa, M., Vidosavljevic, A., Prats, X., 2025. Hybrid method for holistic air traffic demand and capacity balancing optimisation based on sector complexity. Transport. Res. Part C. 180, 105306. https://doi.org/10.1016/j.trc.2025.105306
Metelli, A.M., 2022. Configurable environments in reinforcement learning: an overview. Special Topics Inf. Technol. , 101–113.
Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning. In: International Conference on Machine Learning. PmLR, pp. 1928–1937.
Nosedal, J., Piera, M.A., Solis, A.O., Ferrer, C., 2015. An optimization model to fit airspace demand considering a spatio-temporal analysis of airspace capacity. Transport. Res. Part C. 61, 11–28.
Padakandla, S., 2021. A survey of reinforcement learning algorithms for dynamically varying environments. ACM Comput. Surv. (CSUR) 54 (6), 1–25.
Schäfer, M., Strohmeier, M., Lenders, V., Martinovic, I., Wilhelm, M., 2014. Bringing up opensky: a large-scale ADS-b sensor network for research. In: IPSN-14 Proceedings of the 13th International Symposium on Information Processing in Sensor Networks. IEEE, pp. 83–94.
Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
Schultz, M., Gerdes, I., Standfuß, T., Temme, A., 2019. Future airspace design by dynamic sectorization. In: Air Traffic Management and Systems III: selected Papers of the 5th ENRI International Workshop on ATM/CNS (EIWAC2017) 5. Springer, pp. 19–34.
Sergeeva, M., Delahaye, D., Mancel, C., Vidosavljevic, A., 2017. Dynamic airspace configuration by genetic algorithm. J. Traffic Transp. Eng. (Engl. ed.) 4 (3), 300–314.
Signoriello, A.L., 2017. periodic_voronoi – github repository. [Online]. Available: https://github.com/alsignoriello/periodic_voronoi.
Suarez, N., López, P., Puntero, E., Rodriguez, S., 2014. Quantifying air traffic controller mental workload. Fourth SESAR Innov. Days 220.
Sutton, R.S., Barto, A.G., et al., 1998. Reinforcement Learning: An Introduction. Vol. 1. MIT press Cambridge.
Tang, J., Alam, S., Lokan, C., Abbass, H.A., 2012. A multi-objective approach for dynamic airspace sectorization using agent based and geometric models. Transport. Res. Part C. 21 (1), 89–121. https://doi.org/10.1016/j.trc.2011.08.008
Tien, S.-L., 2010. Demand-responsive airspace sectorization and air traffic controller staffing.
Tobaruela, G., Schuster, W., Majumdar, A., Ochieng, W.Y., Martinez, L., Hendrickx, P., 2014. A method to estimate air traffic controller mental workload based on traffic clearances. J. Air Transp. Manag. 39, 59–71.
Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., De Cola, G., Deleu, T., Goulao, M., Kallinteris, A., Krimmel, M., KG, A., et al., 2024. Gymnasium: a standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032.
Trandac, H., Baptiste, P., Duong, V., 2005. Airspace sectorization with constraints. RAIRO-Operations Research-Recherche Opérationnelle 39 (2), 105–122.
Wong, C. S.Y., Sundaram, S., Sundararajan, N., 2018. Cdas: a cognitive decision-making architecture for dynamic airspace sectorization for efficient operations. IEEE Trans. Intell. Transp. Syst. 20 (5), 1659–1668.
Wong, C. S.Y., Suresh, S., Sundararajan, N., 2020. A rolling horizon optimization approach for dynamic airspace sectorization. IFAC J. Syst. Control 11, 100076.
Wu, G., Fan, M., Shi, J., Feng, Y., 2021. Reinforcement learning based truck-and-drone coordinated delivery. IEEE Trans. Artif. Intell. 4 (4), 754–763.
Xu, Q., Pang, Y., Liu, Y., 2024. Dynamic airspace sectorization with machine learning enhanced workload prediction and clustering. J. Air Transp. Manag. 121, 102683.
Xu, Y., Dalmau, R., Melgosa, M., Montlaur, A., Prats, X., 2020a. A framework for collaborative air traffic flow management minimizing costs for airspace users: enabling trajectory options and flexible pre-tactical delay management. Transp. Res. Part B 134, 229–255. https://doi.org/10.1016/j.trb.2020.02.012
Xu, Y., Prats, X., Delahaye, D., 2020b. Synchronised demand-capacity balancing in collaborative air traffic flow management. Transport. Res. Part C. 114, 359–376. https://doi.org/10.1016/j.trc.2020.02.007
Yazgan, E., Sert, E., Şimşek, D., 2021. Overview of studies on the cognitive workload of the air traffic controller. Int. J. Aviat. Sci. Technol. 2 (01), 28–36.
Zhao, J., Mao, M., Zhao, X., Zou, J., 2020. A hybrid of deep reinforcement learning and local search for the vehicle routing problems. IEEE Trans. Intell. Transp. Syst. 22 (11), 7208–7218.
Zhao, T., Escribano, J., Majumdar, A., Ochieng, W.Y., 2026. Robust 3d dynamic airspace sectorization: a multilayer graph-based approach. J. Air Transp. Manag. 132, 102953.
Zhou, W., Pham, D.-T., Alam, S., 2023. Airfusion: a machine learning framework for balancing air traffic demand and airspace capacity through dynamic airspace sectorization. In: 2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC). IEEE, pp. 5324–5331.